

PRÁCTICA 4 — DESTRUCTORES EN JAVA Y ANALIZADOR DE PALABRAS

¿QUÉ PROBLEMAS TRABAJAMOS?

INTEGRANTE 1 — PONENTE PRINCIPAL

En esta práctica trabajamos dos problemas que al principio parecen bastante diferentes, pero los dos tienen que ver con entender y aprovechar herramientas que Java ya nos ofrece.

Primero investigamos qué pasa con los objetos cuando dejamos de utilizarlos y por qué Java no maneja destructores como otros lenguajes, por ejemplo C++. Después llevamos esa idea a algo más práctico: desarrollar un programa capaz de recibir una oración y encontrar las palabras que se repiten.

Para la primera parte, cada integrante consultó a un LLM desde un enfoque diferente. Investigamos los destructores, el Garbage Collector, el método `finalize()` y el manejo de recursos externos. Después comparamos las respuestas y reunimos los puntos principales en una sola explicación.

Y aquí encontramos una diferencia importante: liberar memoria y cerrar un recurso externo no son exactamente lo mismo.

¿QUÉ PASA CON LA MEMORIA?

INTEGRANTE 2

En lenguajes como C++, existen destructores que permiten controlar qué ocurre cuando un objeto deja de utilizarse. En Java funciona diferente.

Aquí aparece el Garbage Collector, o recolector de basura. Cuando un objeto ya no tiene ninguna referencia apuntándolo, puede ser eliminado automáticamente. Dicho de una forma sencilla: Java se encarga de recuperar la memoria de objetos que el programa ya no puede utilizar.

Pero aquí pasa algo importante: el Garbage Collector se encarga de la memoria, no necesariamente de recursos externos. Si estamos trabajando con un archivo, una conexión de red o una base de datos, necesitamos asegurarnos de cerrarlos correctamente.

Para eso existen herramientas como `AutoCloseable` y `try-with-resources`, que permiten ejecutar automáticamente `close()` al terminar de utilizar el recurso, incluso si ocurre una excepción.

¿ENTONCES HAY DESTRUCTORES?

INTEGRANTE 3

También investigamos `finalize()`. Este método se utilizó durante un tiempo como una especie de intento de acercarse al comportamiento de un destructor, pero actualmente está obsoleto.

Entonces, la idea importante no es simplemente decir que “Java no tiene destructores”. Lo importante es entender qué utiliza en su lugar: el Garbage Collector para gestionar automáticamente la memoria y mecanismos como `try-with-resources` para manejar recursos externos.

¿CÓMO ANALIZAMOS UNA ORACIÓN?

INTEGRANTE 1 — PONENTE PRINCIPAL

Con esa parte clara, pasamos al segundo ejercicio. Aquí desarrollamos un programa que recibe una oración y obtiene información de sus palabras: cuántas hay en total, cuántas son diferentes y cuáles aparecen más de una vez.

Pero antes de empezar a contar tenemos un problema. Para nosotros, “Hola”, “hola” y “hola.” representan la misma palabra, pero para el programa son textos diferentes si no hacemos algo antes.

Por eso primero normalizamos la oración. Básicamente, la preparamos para que esas diferencias no afecten el resultado: convertimos todo a minúsculas y utilizamos una expresión regular para eliminar los signos de puntuación. Después separamos el texto limpio en palabras utilizando los espacios.

Ahora sí podemos contar. Para hacerlo utilizamos un HashMap, que guarda información en pares de clave y valor. En nuestro caso, la clave es la palabra y el valor es la cantidad de veces que apareció.

Por ejemplo, si encontramos “hola” por primera vez, se guarda con un valor de uno. Si vuelve a aparecer, no necesitamos guardar otro “hola”; simplemente aumentamos su contador a dos. Repitiendo este proceso podemos obtener la frecuencia de todas las palabras.

Además, como cada clave del HashMap representa una palabra diferente, su tamaño nos permite saber cuántas palabras distintas encontramos. Y si una palabra tiene una frecuencia mayor que uno, sabemos que está duplicada.

Pero el HashMap no mantiene las palabras ordenadas alfabéticamente. Por eso utilizamos un TreeSet: pasamos ahí las claves y este las ordena automáticamente.

Después solo recorremos esas palabras y mostramos las que tengan una frecuencia mayor a uno.

¿CÓMO SABEMOS QUE FUNCIONA?

INTEGRANTE 4

Para comprobarlo hicimos tres pruebas con situaciones diferentes.

En la primera usamos “Hola mundo. Hola a todos en este mundo!”. Aunque una de las apariciones de “Hola” tiene mayúscula y hay signos de puntuación, el programa las reconoce correctamente como la misma palabra. El resultado fue de 8 palabras totales, 6 distintas y dos duplicadas: “hola” y “mundo”.

En la segunda utilizamos una oración con todavía más signos de puntuación y repeticiones. El programa contó 12 palabras, 7 distintas y detectó tres palabras duplicadas: “el”, “perro” y “rápido”.

¿Y SI NO HAY DUPLICADOS?

INTEGRANTE 5

También necesitábamos comprobar el caso contrario: una oración donde ninguna palabra estuviera repetida.

Para eso utilizamos “Programar en Java es muy interesante.” El programa contó 6 palabras, todas diferentes, y mostró el mensaje “No hay palabras duplicadas”.

Así comprobamos que el programa no solo encuentra repeticiones, sino que también reconoce correctamente cuando simplemente no existen.

¿QUÉ NOS DEJÓ LA PRÁCTICA?

INTEGRANTE 1 — PONENTE PRINCIPAL

Al final, los dos ejercicios nos permitieron cumplir los objetivos de la práctica.

Con la investigación entendimos que Java separa dos cosas que pueden confundirse fácilmente: la memoria de los objetos, que puede gestionar automáticamente el Garbage Collector, y los recursos externos, que necesitan mecanismos específicos para cerrarse correctamente.

Y con el analizador llevamos varios conceptos a un problema concreto. Limpiamos y separamos una oración, usamos un HashMap para contar cuántas veces aparece cada palabra y un TreeSet para mostrarlas ordenadas.

Las pruebas confirmaron que el programa ignora correctamente las diferencias entre mayúsculas, minúsculas y signos de puntuación, y que puede identificar tanto las palabras distintas como las duplicadas.